

A dark green square containing the white text "JS".

ENGINEERING DOCUMENTATION · WRITTEN FOR A FIRST-TIME READER

JanSarthi AI Project Overview

A ground-up walkthrough of what this codebase is, how its pieces fit together, and what a first-time reader needs to know — including the real, current AI architecture (Ask Sarthi, LangGraph, retrieval-augmented generation).

Live	jansarthi-ai.duckdns.org
Stack	FastAPI · SQLite · React + TypeScript · Sarvam AI · LangGraph · ChromaDB · GCP
Prepared	2026-08-30

What JanSarthi AI actually is

A multilingual civic-grievance platform — still built around the same core problem the project started with: the citizen reporting a problem and the municipal worker fixing it often don't share a language. What's changed since the earliest version is **how** that gap gets closed: not a translated web form, but **Ask Sarthi**, a single conversational AI agent a citizen can speak or type to, in any of 6 Indian languages (English, Hindi, Marathi, Odia, Gujarati, Bengali), to file a complaint, ask a civic question, or check a complaint's status.

Under the hood this is a real production system, not a prototype: a stateless **LangGraph** orchestration graph routes every message, a **RAG** (retrieval-augmented generation) pipeline answers civic questions only from a real, sourced knowledge base (never a guess), and hand-rolled guardrails scan every message in and every response out for prompt-injection attempts. **Sarvam AI** (speech-to-text, translation, chat) and **Google Gemini** (photo captioning, with a local model as an automatic fallback) are the two external AI vendors; everything else — the graph, the retrieval logic, the guardrails — is this codebase's own.

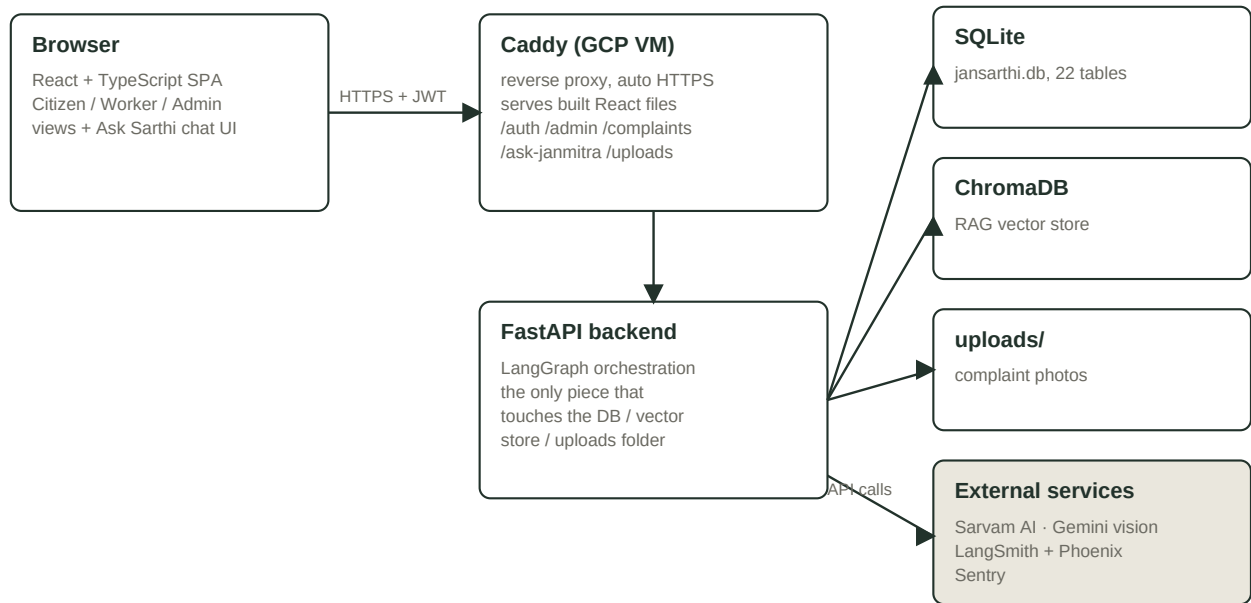
The three kinds of people who use it

ROLE	WHO THEY ARE	WHAT THEY CAN DO	HOW THEIR ACCOUNT IS CREATED
CITIZEN	A resident reporting a problem or asking a civic question	Talk to Ask Sarthi (voice/text/photo) to file complaints or get grounded answers; track their own complaints	Signs up themselves, with mandatory email OTP verification
WORKER	Municipal staff for one specific ward	See every complaint assigned to their ward; accept, reject (auto-reassigns), mark in-progress/resolved	Created only by an Admin — cannot self sign-up
ADMIN	Municipal office overseeing all wards	See every complaint everywhere, manage workers, review AI cost/latency and knowledge-base gaps	Seeded directly into the database via a script — never created through the app itself

That last row is still deliberate, not an oversight: no button, form, or API endpoint anywhere in this codebase can create an Admin. The first Admin account for a real deployment is planted straight into the database via `scripts/seed_admin.py`. “Can an attacker escalate their own privileges through the API” has a provably-no answer, because the capability doesn't exist in the API surface at all.

System architecture

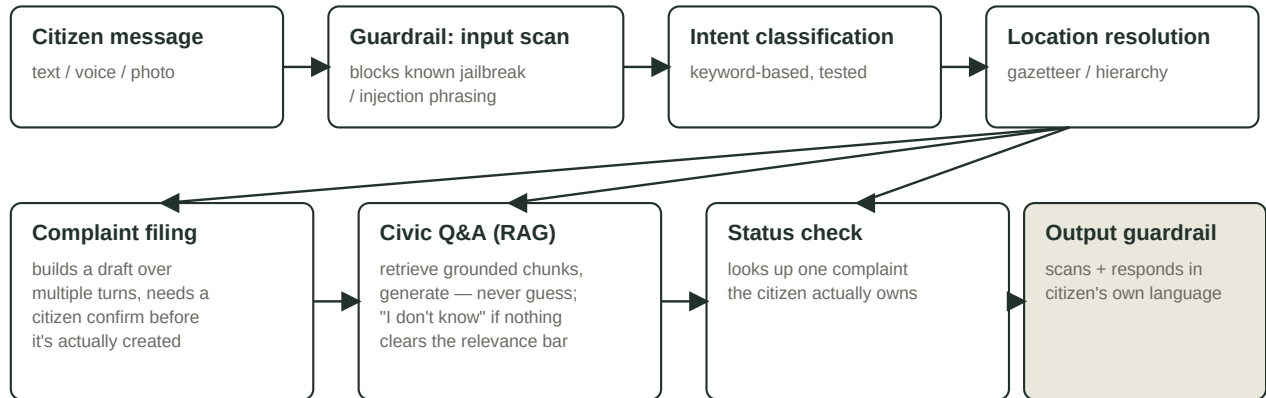
Everything runs on a single GCP VM: two Docker containers (the built React app served by Caddy, and the FastAPI backend) with Caddy as the one thing exposed to the internet. Four external services sit outside this codebase entirely, each reached over the internet with its own API key: Sarvam AI, Google Gemini, LangSmith + Arize Phoenix (tracing), and Sentry (error monitoring).



What this shows: the browser only ever talks to Caddy, which either serves the built frontend directly or reverse-proxies to the backend. The backend is still the only piece that touches SQLite, ChromaDB, or the uploads folder — the browser is never trusted with direct access to any of them, or to an AI API key.

How one message actually flows through Ask Sarthi

Every message, regardless of what it's about, passes through the same shared guardrail input scan and intent classification before the graph decides which of three real jobs it's doing. Filing a complaint is the one path with a hard rule: nothing is written to the database until the citizen explicitly confirms the draft — Ask Sarthi never files a complaint a citizen didn't actually agree to.



Translation, concretely: a complaint's `original_text` is never modified after creation; a canonical English `translated_text` is built once at submit time. Viewing a complaint in a different language translates **on first read, then caches the result** (a real cache table, not a translate-on-every-view design) — the same pattern used for a worker's own progress notes, whose source language is auto-detected rather than assumed from their profile.

Words used throughout this document

TERM	MEANING
LangGraph	The state-machine framework Ask Sarthi's whole conversation flow is built on — 14 real nodes, conditional-edge routing, one shared response node.
RAG	Retrieval-Augmented Generation: look up real, relevant knowledge-base chunks first, then generate an answer grounded only in those chunks — never a guess from the model's own training.
Vector store / ChromaDB	A database that finds chunks of text by meaning (via embeddings), not exact keyword match. This project's RAG knowledge base is indexed here.
Embedding	A numeric representation of a piece of text, positioned so semantically similar text ends up numerically close — what a vector search actually compares.
Reranker	A second, more precise model that re-scores an initial shortlist of retrieved chunks — slower per-item than the first-pass search, so only run on a small shortlist.
Guardrails	Hand-rolled pattern-based checks scanning messages in and responses out for known prompt-injection/jailbreak phrasing — a real floor, not a semantic guarantee.
MCP	Model Context Protocol — a standard way to expose a service's own functions as callable "tools" for an external AI client. This app's services are exposed this way for any MCP-compatible client.
JWT	JSON Web Token — a signed, tamper-proof string proving who a user is on every request, without the server needing to remember sessions.
CSRF	Cross-Site Request Forgery — the attack a cookie-based session opens up; defended here with a double-submit cookie a forged cross-site request can't replicate.
ORM	Object-Relational Mapper — lets Python code describe database rows as regular classes instead of writing raw SQL. This project uses SQLAlchemy.
OTP	One-Time Password — a short emailed code used here to verify a citizen's email address and to authorize a password reset.

The shape of the codebase today

The codebase has grown a great deal since the earliest version — 6 backend route modules and 30+ services, 21 frontend pages and 36+ components. Rather than hand-maintain an exhaustive file list here (a list like that has already drifted stale twice in this project's history), this section names the load-bearing pieces worth knowing by name on a first read; the real, current folders are the source of truth.

Backend (backend/) — highlights

PIECE	WHAT IT'S FOR
routes/ask_janmitra.py	The Ask Sarthi endpoint — the entry point into the LangGraph orchestration
services/orchestration/graph.py, nodes.py	The 14-node LangGraph graph and every node's real logic
services/rag_retriever.py, vector_store.py	Embed → search → hybrid BM25 widen → threshold → rerank → tie-break
services/guardrails.py	Prompt-injection input/output scanning, shared by every flow
services/assignment_service.py	Ward-based worker assignment, with automatic reassignment on rejection
services/complaint_agent.py	The original AI pipeline: STT → normalize → translate → summarize → save
services/vision_service.py	Photo captioning — Gemini first, a local model as automatic fallback
services/observability/tracing.py	LangSmith + Phoenix dual tracing, real ■ cost tracking
repositories/complaint_repository.py	Plain, reviewable DB queries — never LLM-constructed SQL, anywhere
scripts/build_rag_knowledge_base.py, build_rag_embeddings.py	Regenerate the RAG chunk file and the ChromaDB index
scripts/seed_admin.py	Plants the first Admin account directly into the database

Frontend (frontend-react/src/) — highlights

PIECE	WHAT IT'S FOR
pages/AskJanMitra.tsx	The actual Ask Sarthi chat UI — citizens' main way of using the app
pages/{Citizen,Worker,Admin}Dashboard.tsx	The three role-scoped dashboards
pages/{Citizen,Worker,Admin}ComplaintDetail.tsx	Full complaint detail, including the location breadcrumb and PDF report
components/LocationPicker.tsx	GPS or manual location capture, with a live reverse-geocoded preview
components/PasswordInput.tsx	Shared show/hide password toggle, used across every auth form
lib/api.ts	Every backend call the frontend makes, in one file, with auth + CSRF handling

PIECE	WHAT IT'S FOR
lib/auth.tsx	Session state from httpOnly cookies — never localStorage for tokens
lib/useAudioRecorder.ts	Client-side voice chunking (≤ 28 s pieces) so long recordings never hit Sarvam's 30s cap

The database, in plain terms

Still SQLite — one file on disk, no separate server process. What's changed is scale: this started as 4 tables (users, complaints, complaint_rejections, complaint_translations) and has grown to **22**, as the app grew a real location hierarchy, a fuller complaint lifecycle, notifications, AI observability, and auth.

TABLE GROUP	WHAT IT ADDED
Original core (4 tables)	Users (single-table, role-discriminated), complaints (original + translated text kept side by side, permanently), rejections, a translation cache
Location hierarchy (7 tables)	A real, ID-based state → district → sub-district → ulb → zone → ward → locality chain, so a ward can be referenced by a stable ID, not just free text
Fuller complaint lifecycle	A status-change audit trail, worker progress notes + their cached translations, multi-photo evidence
Notifications & AI observability	In-app alerts, an AI-answer cache, per-request cost/latency logs and alert state
Auth	Refresh-token rotation records, email OTP codes for verification and password reset

There is still no migration framework (no Alembic) — a known, honest gap. Adding a column to an existing table with real data needs a small, one-off manual script; only a brand-new table gets auto-created on startup.

Authentication & roles, step by step

The token itself is still a hand-rolled JWT (HS256, against Python's standard library only — no third-party JWT package, since this app only ever verifies tokens it issued itself). What's changed is where it lives and what protects it.

THEN	NOW
Token kept in localStorage, sent as a Bearer header	Token kept in an httpOnly cookie — unreadable to any script on the page, including one injected via XSS
No CSRF concern (no cookie to forge)	A cookie needs its own defense: a double-submit CSRF cookie a forged cross-site request can't replicate
One token, no refresh	Access + refresh tokens; refresh rotates on every use, and a reused (already-rotated) refresh token revokes every active session for that user
No email/phone verification	Mandatory email OTP verification at signup; a real password-reset flow, also via emailed OTP
No rate limiting	4 independent sliding-window limiters (login, signup, AI requests, a general baseline) plus a separate OTP limiter

Authorization is still enforced two ways: route-level (`require_role(...)` blocks an entire endpoint before its logic runs) and row-level (an explicit ownership check confirms a specific record belongs to the caller, not just that their role is generally allowed).

Three real journeys through the app

A. A citizen files a complaint through Ask Sarthi

Opens Ask Sarthi, types or speaks a problem in Marathi. Ask Sarthi classifies the intent as a complaint, resolves the location (asking a clarifying question if it can't), builds a draft, and explicitly asks the citizen to confirm before anything is saved. Once confirmed, the complaint is created, translated to English for storage, and immediately handed to `assign_next_worker()` for that ward.

B. A worker rejects, and it reassigns itself

A worker opens their ward queue, sees a new complaint with its short AI summary above the full text, and rejects it (every ward has at least 2 workers for exactly this reason). The assignment service automatically routes it to the next eligible worker in that ward — no admin intervention needed, and the same worker can never reject the same complaint twice.

C. An admin reviews AI cost and a knowledge gap

Logs into the Admin AI Monitoring page, sees real ■ cost by model (computed from Sarvam's own published pricing, not estimated), and follows a “High AI latency” alert straight to the specific slow requests behind it. Separately, reviews a citizen question the knowledge base couldn't answer, flagged automatically for review.

What's real, and what's still open

22 database tables, still SQLite	1,057 RAG chunks indexed (609 VERIFIED)	≤28s voice chunk length — chunked client-side	4 independent rate limiters
--	---	---	---

Real, open gaps — named on purpose rather than left implicit:

GAP	STATUS
PostgreSQL	Still SQLite — the top deferred production gap; mostly a DATABASE_URL change thanks to the ORM
Category detection via LLM	Evaluated live, declined on measured evidence (4/5 misclassified, including a confidently-wrong out-of-scope call) — stays a deterministic keyword classifier
Priority detection	Not evaluated yet
Tool calling (LLM-driven)	Routing is deterministic intent classification by design, not an LLM deciding at runtime
Redis, async queues	Not used anywhere in this codebase yet
Prometheus / Grafana / Alertmanager	Not used; a lighter in-app substitute exists instead (two admin alerts computed from the last 20 real requests)
Centralized logging (beyond errors)	Sentry centralizes errors specifically, not all application logs
Phone number verification	Only email is verified via OTP; a citizen's phone number itself is still never confirmed as theirs

Running the whole thing locally

1. Backend

```
pip install -r requirements.txt
cp .env.example .env # fill in SARVAM_API_KEY, JWT_SECRET_KEY (any long random string)
python scripts/build_rag_knowledge_base.py # generates chunks.json from knowledge_records/
python scripts/build_rag_embeddings.py # populates the real ChromaDB index
uvicorn backend.main:app --reload # API docs at localhost:8000/docs
python scripts/seed_admin.py --phone 9999999999 --password "change-me" --name "Your Name"
```

2. Frontend

```
cd frontend-react && npm install
npm run dev # opens at localhost:5173
```

3. Try the three roles

Sign up as a citizen through the app itself (verify the email OTP). Log in as the admin you just seeded, add a worker for a ward, then log in as that worker in a different browser (or incognito window) to see both sides at once. Open Ask Sarthi as the citizen and try filing a complaint, asking a civic question, and checking a complaint's status.

How this is tested

Two separate test suites, for two separate layers, both run automatically on every push via GitHub Actions before anything can reach production:

SUITE	WHAT IT COVERS
Backend (pytest tests/ -v)	The full suite — auth, complaints, assignment, the whole Ask Sarthi/RAG pipeline, guardrails, tracing. All external AI calls are mocked except a small, deliberate set that hit the real Sarvam API to prove real behavior (e.g. language auto-detection).
Frontend (Playwright, frontend-react/e2e/)	Drives a real browser through language selection, signup/login, the full complaint lifecycle (submit → assign → reject → reassign → accept → resolve → feedback), and voice-recording UI.

CI (ci.yml) gates CD (cd.yml): a merge to main only reaches the live GCP server after both the backend test suite and a real frontend production build pass — see docs/CI_CD_GITHUB_ACTIONS.md for the full pipeline.